



UNIVERSIDADE PRESBITERIANA MACKENZIE

Faculdade de Computação e Informática

Sistema Inteligente de Monitoramento de Lixeiras Urbanas Utilizando Internet das Coisas (IoT)

Cauê Tanigawa Paganeli, Matheus Hypolito, Rodrigo Soriani, Leandro Carlos Fernandes

¹Universidade Presbiteriana Mackenzie (UPM)

Rua da Consolação, 930 – Consolação, São Paulo – SP, 01302-907 – Brazil

10310783@mackenzista.com.br, 10443058@mackenzista.com.br,

10433736@mackenzista.com.br, fci.ead.adsistemas@mackenzie.br

Abstract. The growth of urban populations has increased the challenges related to waste management in cities. Inefficient waste collection can lead to overflowing trash bins, environmental pollution and public health issues. In this context, Internet of Things (IoT) technologies have enabled the development of intelligent systems capable of monitoring urban infrastructure in real time.

This paper presents the development and implementation of a smart waste bin monitoring prototype using a NodeMCU ESP8266, an HC-SR04 ultrasonic sensor, a LED actuator, MQTT communication and a Node-RED dashboard. The system measures the distance between the ultrasonic sensor and the waste level inside the bin, estimates the filling percentage and classifies the bin as empty, partially filled or full.

The collected data are transmitted through Wi-Fi using the MQTT protocol and published to a broker. The Node-RED platform subscribes to the MQTT topic, processes the received JSON messages and displays the information in a real-time dashboard. The final prototype demonstrates the integration between sensing, embedded processing, local actuation and remote monitoring, contributing to smart city applications and supporting Sustainable Development Goal 11, related to sustainable cities and communities (UNITED NATIONS, 2015).

Resumo. O crescimento da população urbana tem ampliado os desafios relacionados à gestão de resíduos sólidos nas cidades. Sistemas tradicionais de coleta de lixo operam frequentemente com rotas fixas, o que pode resultar em lixeiras transbordando em algumas regiões, enquanto outras são coletadas antes da necessidade real. Nesse contexto, tecnologias baseadas em Internet

das Coisas (IoT) têm possibilitado o desenvolvimento de soluções inteligentes para o monitoramento da infraestrutura urbana.

Este trabalho apresenta o desenvolvimento e a implementação de um protótipo de lixeira inteligente utilizando NodeMCU ESP8266, sensor ultrassônico HC-SR04, LED como atuador, comunicação MQTT e dashboard no Node-RED. O sistema mede a distância entre o sensor e o nível de resíduos no interior da lixeira, estima o percentual de preenchimento e classifica o estado da lixeira como vazia, parcialmente preenchida ou cheia.

Os dados coletados são transmitidos via Wi-Fi por meio do protocolo MQTT e publicados em um broker. A plataforma Node-RED recebe as mensagens MQTT, processa os dados em formato JSON e apresenta as informações em um dashboard em tempo real. O protótipo final demonstra a integração entre sensoramento, processamento embarcado, atuação local e monitoramento remoto, contribuindo para aplicações de cidades inteligentes e alinhando-se ao Objetivo de Desenvolvimento Sustentável 11, voltado a cidades e comunidades sustentáveis (UNITED NATIONS, 2015).

1. Introdução

O crescimento das áreas urbanas e o aumento da geração de resíduos sólidos representam desafios significativos para a gestão das cidades contemporâneas. A coleta e o gerenciamento inadequados de resíduos podem provocar impactos ambientais, degradação de espaços públicos e riscos à saúde da população. Nesse cenário, torna-se fundamental buscar soluções tecnológicas que contribuam para tornar os serviços urbanos mais eficientes, sustentáveis e compatíveis com as demandas de cidades em constante expansão (UNITED NATIONS, 2015). Com o avanço das tecnologias digitais, a Internet das Coisas (IoT) tem se destacado como uma das principais abordagens para o desenvolvimento de cidades inteligentes. A IoT possibilita a conexão de dispositivos físicos à internet, permitindo a coleta, o processamento e o compartilhamento de dados em tempo real por meio de sensores, microcontroladores e plataformas de monitoramento. Esses sistemas permitem acompanhar diferentes aspectos do ambiente urbano e apoiar a tomada de decisão baseada em dados, característica essencial em aplicações de infraestrutura inteligente (SANTOS et al., 2016).

Diversas aplicações de IoT têm sido desenvolvidas para o monitoramento de ambientes e a automação de processos. Sistemas baseados em sensores podem coletar informações do ambiente e enviá-las para plataformas online, permitindo o acompanhamento remoto das condições monitoradas. Tecnologias semelhantes já são utilizadas em sistemas automatizados voltados ao monitoramento de variáveis ambientais e ao apoio à tomada de decisão a partir de dados coletados por sensores (SEENU; MOHAN; JEEVANATH, 2018). De modo semelhante, aplicações voltadas à agricultura inteligente demonstram como sensores conectados à internet podem ser empregados para ampliar a eficiência operacional e o uso racional de recursos (COELHO et al., 2020).

Além disso, soluções baseadas em sensores e microcontroladores vêm sendo aplicadas em diferentes sistemas de monitoramento remoto, permitindo a transmissão de dados para plataformas de supervisão e análise em tempo real. Tais soluções envolvem a integração entre aquisição de dados, processamento embarcado e comunicação em rede, características centrais de sistemas IoT e de sistemas embarcados conectados (ANANDRAVISEKAR et al., 2018; OLIVEIRA; ANDRADE, 2009).

Embora existam aplicações consolidadas de IoT em áreas como agricultura, automação e vigilância remota, observa-se a necessidade de adaptar esses conceitos para problemas especificamente urbanos, como a gestão de resíduos sólidos em espaços públicos. Nesse contexto, o monitoramento do nível de preenchimento de lixeiras representa uma aplicação aderente às demandas de cidades inteligentes, pois permite decisões operacionais baseadas em dados, melhora a eficiência da coleta e contribui para a redução de impactos ambientais associados ao transbordamento de resíduos (SANTOS et al., 2016; UNITED NATIONS, 2015).

Nesse cenário, este trabalho propõe o desenvolvimento de um sistema inteligente de monitoramento de lixeiras urbanas utilizando Internet das Coisas. O sistema foi composto por sensores capazes de medir o nível de resíduos no interior das lixeiras e enviar essas informações, por meio de comunicação em rede, para uma plataforma online de monitoramento. A partir desses dados, torna-se possível identificar com maior precisão quando as lixeiras atingem níveis críticos de preenchimento, permitindo otimizar as rotas de coleta de resíduos e melhorar a eficiência dos serviços urbanos.

Com isso, espera-se contribuir para uma gestão mais eficiente da limpeza urbana, reduzindo o transbordamento de resíduos em áreas públicas e apoiando iniciativas voltadas ao desenvolvimento de cidades mais sustentáveis, em consonância com os princípios estabelecidos pelo Objetivo de Desenvolvimento Sustentável 11 (UNITED NATIONS, 2015).

Além da fundamentação apresentada, esta versão final do trabalho avança da proposta conceitual para a implementação prática do protótipo. O sistema foi efetivamente montado e testado utilizando uma placa NodeMCU ESP8266, um sensor ultrassônico HC-SR04, um LED como atuador, comunicação MQTT e um dashboard desenvolvido no Node-RED.

Essa implementação permite demonstrar, na prática, o ciclo completo de uma solução IoT: aquisição de dados físicos por sensor, processamento embarcado, acionamento de atuador, transmissão por protocolo de comunicação e visualização remota das informações. Dessa forma, o projeto não se limita à descrição teórica de uma aplicação de cidade inteligente, mas apresenta uma solução funcional de monitoramento de lixeira em tempo real.

2. Materiais e Métodos

Este trabalho propõe o desenvolvimento de um sistema inteligente de monitoramento de lixeiras urbanas utilizando Internet das Coisas (IoT), com capacidade de identificar o nível de preenchimento e sinalizar quando for necessária a coleta.

O sistema foi baseado em um microcontrolador responsável pela leitura de sensores, processamento dos dados e acionamento de um atuador. A arquitetura segue o modelo de objetos inteligentes, integrando sensores, unidade de processamento, comunicação e atuação, conforme a estrutura típica de sistemas embarcados aplicados à IoT (SANTOS et al., 2016).

2.1 Plataforma de prototipagem

A plataforma utilizada para o desenvolvimento do projeto foi o NodeMCU (ESP8266), amplamente empregado em aplicações de Internet das Coisas devido à sua capacidade de integrar processamento e conectividade Wi-Fi em uma única solução.

O NodeMCU é baseado no microcontrolador ESP8266, que possui arquitetura de 32 bits, entradas e saídas digitais, além de suporte à comunicação por rede sem fio. Essa característica

permite a integração direta com serviços de comunicação em nuvem, eliminando a necessidade de módulos adicionais para conectividade (NODEMCU TEAM, s.d.).

Além disso, o NodeMCU possibilita a execução de firmware responsável pela leitura de sensores, processamento dos dados e envio das informações para sistemas externos, característica essencial em sistemas embarcados conectados (SANTOS et al., 2016).

A escolha dessa plataforma se justifica pela necessidade de comunicação direta com a internet e pela possibilidade de execução de firmware compatível com bibliotecas de comunicação MQTT, requisito importante para aplicações IoT, além da simplicidade de implementação e compatibilidade com ambientes de desenvolvimento como a Arduino IDE. Adicionalmente, a utilização do NodeMCU reduz a complexidade da arquitetura do sistema, uma vez que integra processamento e comunicação em um único dispositivo (NODEMCU TEAM, s.d.; SANTOS et al., 2016).

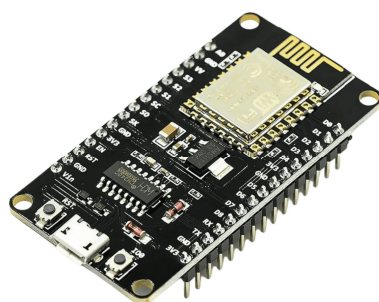


Figura 1 – Placa NodeMCU ESP8266.

Fonte: NodeMCU ESP8266 (2020).

2.2 Sensor de nível de resíduos

Para a medição do nível de preenchimento da lixeira, o sistema considerou a utilização de um sensor de distância como elemento principal de aquisição de dados, sendo o sensor ultrassônico HC-SR04 uma solução adequada para essa aplicação.

O funcionamento desse sensor baseia-se na emissão de ondas ultrassônicas e na medição do tempo necessário para o retorno do sinal refletido. A partir desse tempo, é possível calcular a distância entre o sensor e a superfície dos resíduos. Considerando a altura interna útil da lixeira, essa distância pode ser convertida em uma estimativa do nível de ocupação. Essa abordagem

permite a utilização de sensores indiretos para estimar grandezas físicas, estratégia comum em sistemas embarcados com restrições de custo e complexidade (OLIVEIRA; ANDRADE, 2009).

No contexto deste trabalho, a variável de interesse, correspondente ao nível de preenchimento, não é medida diretamente, sendo inferida a partir da distância obtida. Essa abordagem permite representar o estado da lixeira de forma contínua, possibilitando a definição de diferentes níveis operacionais.

Do ponto de vista de integração elétrica, destaca-se a necessidade de compatibilização entre os níveis de tensão do sensor e do microcontrolador. O HC-SR04 é usualmente empregado com sinal de saída incompatível diretamente com a entrada lógica de 3,3 V do ESP8266, exigindo cuidados adicionais na interface dos sinais digitais, especialmente no pino ECHO (HC-SR04 ULTRASONIC SENSOR DATASHEET, 2026).



Figura 2 - Sensor Ultrassônico HC-SR04.

Fonte: ROBOCORE (2026).

2.3 Atuador

Como atuador do sistema, foi utilizado um LED, responsável por indicar quando a lixeira atinge um nível crítico de preenchimento.

Os atuadores são dispositivos responsáveis por executar ações no ambiente com base nos dados processados pelo sistema, permitindo a interação entre o processamento digital e o ambiente físico monitorado (BYERS, s.d.). Neste projeto, o LED foi utilizado como sinalizador visual

local, sendo acionado sempre que o nível de preenchimento ultrapassou um limite crítico previamente definido, indicando a necessidade de coleta.

Para proteção do componente e limitação da corrente elétrica, foi utilizado um resistor de 220Ω em série com o LED indicador.



Figura 3 - LED.

Fonte: MAKERHERO (2026).

2.4 Comunicação e protocolo MQTT

A comunicação dos dados foi realizada por meio do protocolo MQTT (Message Queuing Telemetry Transport), amplamente utilizado em aplicações de Internet das Coisas devido à sua leveza e eficiência (IBM, s.d.; MQTT.ORG, 2026).

O NodeMCU atuou como cliente MQTT, sendo responsável por estabelecer conexão com um broker, como o Eclipse Mosquitto ou outro serviço compatível, e publicar os dados coletados em tópicos específicos.

Esse modelo de comunicação, baseado no padrão *publish/subscribe*, permite que diferentes aplicações possam acessar as informações em tempo real, sem necessidade de comunicação direta entre os dispositivos. Essa característica favorece a escalabilidade da solução e reduz o acoplamento entre o nó sensor e as aplicações de supervisão (IBM, s.d.).

A utilização do protocolo MQTT se justifica pelo baixo consumo de banda, simplicidade de implementação e alta eficiência em ambientes com restrições de recursos, características importantes em sistemas embarcados conectados (IBM, s.d.; SANTOS et al., 2016).

Os dados podem ser organizados em tópicos estruturados por identificação da lixeira, permitindo escalabilidade da solução. O uso de níveis de Qualidade de Serviço (QoS) possibilita ajustar a confiabilidade da entrega das mensagens conforme a criticidade da aplicação.

2.5 Plataforma de monitoramento

Os dados coletados pelo sistema foram disponibilizados em plataformas IoT compatíveis com MQTT, permitindo o acompanhamento remoto das condições das lixeiras.

Essa abordagem possibilita a análise de dados em tempo real e o acompanhamento contínuo das condições operacionais, contribuindo para a tomada de decisão baseada em informações coletadas por sensores conectados (SANTOS et al., 2016).

Além disso, a utilização de plataformas de monitoramento permite a visualização dos dados em diferentes dispositivos, como computadores e dispositivos móveis. Dessa forma, pode-se acompanhar o percentual estimado de ocupação da lixeira, o estado do sistema e a ocorrência de alertas críticos, informações relevantes para o planejamento da coleta urbana.

2.6 Metodologia de funcionamento

O funcionamento do sistema inicia-se com o acionamento do sensor ultrassônico HC-SR04, responsável pela medição da distância entre o sensor e o nível de resíduos presente na lixeira. O sinal de disparo (TRIG) é enviado pelo microcontrolador, enquanto o sinal de retorno (ECHO) é recebido e processado para o cálculo da distância (HC-SR04 ULTRASONIC SENSOR DATASHEET, 2026).

O sinal de ECHO é condicionado por meio de um divisor de tensão composto por resistores de 1 k Ω e 2 k Ω , garantindo a adequação dos níveis de tensão ao padrão lógico do ESP8266. Detalhes da implementação física dessa solução são apresentados na Seção 2.8.

A partir dos valores obtidos, o microcontrolador realiza a análise do nível de preenchimento da lixeira. Considerando-se a altura útil interna do recipiente, a estimativa do nível de ocupação pode ser obtida pela expressão:

$$Nivel(\%) = \left(\frac{H - d}{H} \right) * 100$$

H representa a altura útil interna da lixeira e d corresponde à distância medida entre o sensor e a superfície dos resíduos. Essa estratégia caracteriza o uso de sensoriamento indireto em sistemas embarcados, permitindo a inferência de uma variável de interesse a partir de uma medição física disponível (OLIVEIRA; ANDRADE, 2009).

Com base nesse cálculo, o sistema realiza a tomada de decisão utilizando limites previamente definidos. Quando um limite crítico é atingido, o sistema aciona o LED como forma de sinalização local. Paralelamente, os dados são transmitidos via rede Wi-Fi utilizando o protocolo MQTT, permitindo o monitoramento remoto.

Essa metodologia representa o funcionamento típico de um sistema IoT, no qual há aquisição de dados, processamento embarcado, tomada de decisão e comunicação com sistemas externos (BYERS, s.d.; SANTOS et al., 2016).

Para aumentar a confiabilidade do protótipo, recomenda-se a realização de múltiplas leituras sucessivas com cálculo de média ou mediana, reduzindo oscilações inerentes ao sensoriamento ultrassônico. Também se recomenda a implementação de rotinas de reconexão em caso de falha de comunicação com a rede Wi-Fi ou com o broker MQTT.

2.7 Considerações sobre a integração do sistema

A integração entre os componentes do sistema foi definida considerando a separação entre aquisição de dados, processamento e comunicação, permitindo maior flexibilidade na implementação e validação do sistema.

A utilização de uma variável indireta para inferência do nível de ocupação representa uma abordagem comum em sistemas embarcados, na qual sensores disponíveis são utilizados para estimar grandezas de interesse (OLIVEIRA; ANDRADE, 2009).

Essa abordagem contribui para o desenvolvimento de sistemas mais eficientes, mantendo baixo custo e viabilidade de implementação.

Além disso, a preocupação com a compatibilidade elétrica entre os dispositivos, especialmente em relação aos níveis de tensão, evidencia a aplicação de boas práticas de projeto em sistemas

embarcados, contribuindo para a confiabilidade e robustez da solução proposta (OLIVEIRA; ANDRADE, 2009). O circuito de adaptação de tensão foi implementado utilizando resistores de 1 k Ω e 2 k Ω , formando um divisor de tensão capaz de reduzir o sinal de aproximadamente 5 V proveniente do pino ECHO do sensor HC-SR04 para níveis compatíveis com as entradas digitais de 3,3 V do NodeMCU ESP8266.

A organização modular do sistema, composta por sensoriamento, processamento embarcado, atuação local e comunicação em rede, reforça a aderência do projeto ao conceito de objetos inteligentes conectados aplicados ao contexto de cidades inteligentes (SANTOS et al., 2016).

2.8 Modelo de montagem do protótipo

Para a implementação do protótipo físico, foi desenvolvido um modelo de montagem no software Fritzing, permitindo a representação visual das conexões elétricas entre os componentes do sistema (FRITZING, s.d.).

O microcontrolador NodeMCU (ESP8266) atua como unidade central do sistema, sendo responsável pela leitura do sensor, processamento dos dados e acionamento do atuador. O sensor ultrassônico HC-SR04 foi conectado ao microcontrolador por meio de quatro pinos principais: alimentação, terra (GND), disparo (TRIG) e recepção (ECHO).

O pino TRIG do sensor foi conectado diretamente a uma porta digital do NodeMCU, responsável por enviar o sinal de acionamento do sensor. Já o pino ECHO foi conectado ao microcontrolador por meio de um divisor de tensão composto por dois resistores, com valores de 1 k Ω e 2 k Ω .

O divisor de tensão foi implementado com o objetivo de reduzir o nível do sinal de saída do sensor, garantindo compatibilidade com os níveis lógicos do ESP8266. A configuração adotada consiste na utilização de resistores conectados ao sinal de ECHO, formando o ponto de divisão de tensão no pino de entrada do microcontrolador. Essa solução visa preservar a integridade elétrica da interface entre os dispositivos (HC-SR04 ULTRASONIC SENSOR DATASHEET, 2026; OLIVEIRA; ANDRADE, 2009).

O LED foi conectado a uma porta digital do NodeMCU por meio de um resistor limitador de corrente de 220 Ω , sendo utilizado como atuador para indicação do estado da lixeira. Todas as

conexões compartilham uma referência comum de terra, assegurando coerência elétrica ao circuito.

A Figura 4 apresenta o modelo de montagem do protótipo desenvolvido no software Fritzing, evidenciando as conexões entre sensor, microcontrolador e atuador, bem como a implementação do divisor de tensão no sinal de ECHO.

Essa abordagem de montagem evidencia a integração entre hardware e software no contexto de sistemas embarcados conectados, reforçando a aplicação prática dos conceitos de Internet das Coisas no desenvolvimento de soluções voltadas a cidades inteligentes. Do ponto de vista de projeto, a adoção dessas soluções evidencia a preocupação com a integridade elétrica do sistema e com a compatibilidade entre dispositivos com diferentes níveis lógicos, aspecto fundamental no desenvolvimento de sistemas embarcados confiáveis (FRITZING, s.d.; OLIVEIRA; ANDRADE, 2009).

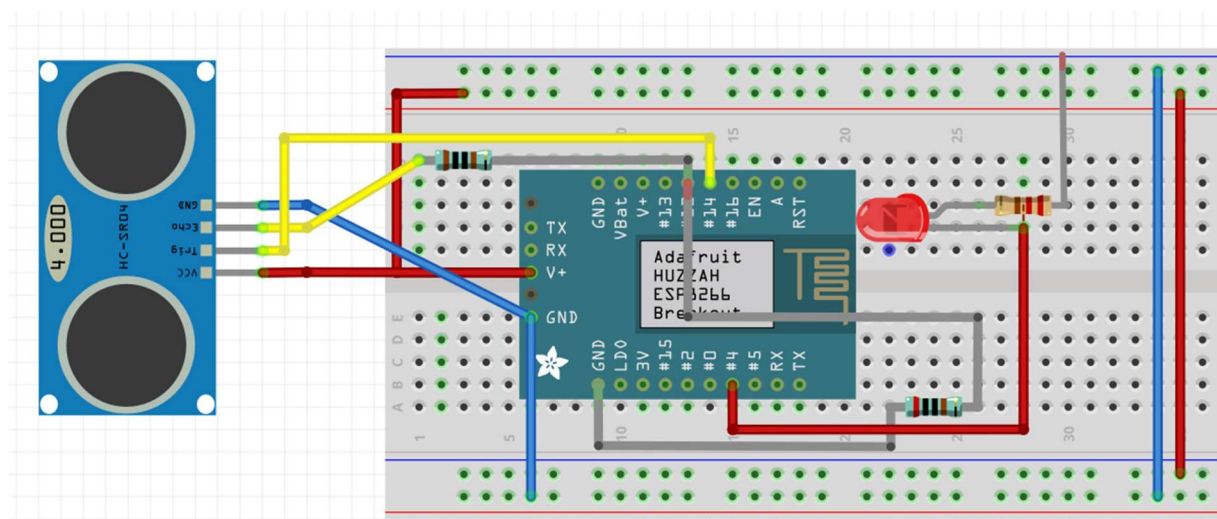


Figura 4 - Modelo de montagem do sistema no software Fritzing.

Fonte: Elaborado pelos autores (2026).

A montagem foi realizada em uma protoboard, permitindo a interligação dos componentes por meio de jumpers e facilitando alterações durante as etapas de desenvolvimento, testes e validação do sistema.

Tabela 1 – Ligações elétricas do protótipo

Componente	Pino do componente	Conexão no NodeMCU ESP8266	Observação
HC-SR04	VCC	VIN / 5V	Alimentação do sensor
HC-SR04	GND	G (GND)	Referência comum
HC-SR04	TRIG	D5	Disparo da medição
HC-SR04	ECHO	D6	Conectado através de divisor de tensão
LED	Ânodo (+)	D2	Controle do LED indicador
LED	Cátodo (-)	G (GND)	Através de resistor de 220 Ω
Resistor 1	1 k Ω	Divisor de tensão	Proteção do pino ECHO
Resistor 2	2 k Ω	Divisor de tensão	Redução de 5 V para aproximadamente 3,3 V

Fonte: Elaborado pelos autores (2026).

2.9 Versão final do protótipo implementado

Na versão final do protótipo, a solução foi implementada utilizando uma placa NodeMCU ESP8266 como unidade principal de controle e comunicação. Essa escolha permitiu integrar diretamente o protótipo à rede Wi-Fi e ao broker MQTT, sem a necessidade de utilizar um Arduino UNO conectado a outro equipamento para intermediar a comunicação com a internet.

O protótipo utiliza um sensor ultrassônico HC-SR04 para medir a distância entre o sensor e o conteúdo interno da lixeira. A partir dessa distância, o sistema interpreta o nível de preenchimento e classifica o estado da lixeira em três categorias: cheia, parcial ou vazia. Quanto menor a distância medida pelo sensor, maior é o nível de preenchimento da lixeira.

Além do sensor, foi utilizado um LED como atuador visual. Esse LED é acionado quando a lixeira atinge o estado de cheia, funcionando como alerta local para indicar que a coleta ou esvaziamento deve ser realizado. Dessa forma, o protótipo atende ao requisito de possuir ao menos um sensor e um atuador integrados à solução.

A comunicação entre o protótipo físico e a interface de monitoramento foi realizada por meio do protocolo MQTT. O NodeMCU publica periodicamente as informações coletadas no tópico MQTT configurado, enviando dados como distância medida em centímetros e status da lixeira. Esses dados são recebidos no Node-RED, onde são tratados e apresentados em um dashboard com indicador de nível, status textual, histórico gráfico e horário da última atualização.

2.10 Comunicação MQTT e integração com o Node-RED

A comunicação do protótipo foi implementada utilizando o protocolo MQTT, baseado no modelo publish/subscribe. Nesse modelo, o NodeMCU atua como publicador das mensagens, enquanto o Node-RED atua como assinante do tópico responsável por receber os dados.

A utilização do protocolo MQTT em conjunto com plataformas embarcadas de baixo custo, como o ESP8266, é amplamente empregada em aplicações de Internet das Coisas devido à sua leveza, eficiência na transmissão de mensagens e baixo consumo de recursos computacionais, características adequadas para sistemas distribuídos de monitoramento em tempo real (NAYYAR; PURI, 2018).

O broker utilizado foi o HiveMQ público, acessado pelo endereço `broker.hivemq.com` na porta 1883. Essa escolha permitiu validar a comunicação via internet utilizando TCP/IP, atendendo ao requisito obrigatório de uso do protocolo MQTT no projeto.

O tópico utilizado para envio das mensagens foi:

`grupo10/lixeria/nivel`

As mensagens foram estruturadas em formato JSON, contendo a distância medida pelo sensor e o status calculado pelo firmware. Um exemplo de mensagem enviada pelo NodeMCU é apresentado abaixo:

```
{"distancia_cm":16.92,"status":"PARCIAL"}
```

O campo `distancia_cm` representa a distância medida em centímetros pelo sensor ultrassônico. O campo `status` representa a classificação definida pelo sistema embarcado, podendo assumir os valores CHEIA, PARCIAL ou VAZIA.

No Node-RED, um nó MQTT IN foi configurado para assinar o tópico `grupo10/lixeria/nivel`. Em seguida, um nó JSON converte a mensagem recebida em objeto manipulável. Após essa conversão, funções específicas tratam os dados para atualização dos elementos visuais do dashboard.

2.11 Software embarcado desenvolvido

O software embarcado foi desenvolvido na Arduino IDE utilizando as bibliotecas ESP8266WiFi.h e PubSubClient.h. A biblioteca ESP8266WiFi.h foi utilizada para conectar o NodeMCU à rede Wi-Fi, enquanto a PubSubClient.h foi utilizada para estabelecer a comunicação com o broker MQTT.

O ambiente Arduino IDE foi escolhido por oferecer uma interface simples para desenvolvimento, compilação e gravação de firmware em dispositivos compatíveis com ESP8266, além de ampla disponibilidade de bibliotecas para aplicações IoT (ARDUINO, 2026).

No código, foram definidos os pinos D5, D6 e D2. O pino D5 foi utilizado para o sinal TRIG do sensor ultrassônico, responsável por disparar o pulso de medição. O pino D6 foi utilizado para o sinal ECHO, responsável por receber o retorno do sensor. O pino D2 foi utilizado para controlar o LED indicador.

O firmware possui uma função de conexão Wi-Fi, responsável por conectar o NodeMCU à rede configurada, e uma função de conexão MQTT, responsável por conectar o dispositivo ao broker. Caso a conexão MQTT seja perdida, o sistema tenta reconectar automaticamente, aumentando a confiabilidade do protótipo.

A função medirDistancia() realiza o acionamento do sensor ultrassônico. O sistema envia um pulso pelo pino TRIG, mede o tempo de retorno pelo pino ECHO e calcula a distância em centímetros. A fórmula utilizada considera a velocidade aproximada do som no ar:

$$\text{distância} = \text{duração} \times 0,034 / 2$$

A divisão por dois ocorre porque o tempo medido corresponde ao percurso de ida e volta do sinal ultrassônico.

Após a leitura da distância, a função classificarStatus() interpreta o valor obtido e define o estado da lixeira conforme a tabela a seguir.

Tabela 2 – Critérios de classificação da lixeira no firmware.

Distância medida	Percentual estimado	Status atribuído
Menor que 10 cm	75% a 100%	CHEIA
De 10 cm até 35 cm	45% a 75%	PARCIAL
Maior que 35 cm	0% a 45%	VAZIA
Leitura inválida	Não aplicável	ERRO_LEITURA

Fonte: Elaborado pelos autores (2026).

No loop principal, o sistema verifica a conexão MQTT, realiza uma nova leitura a cada dois segundos, classifica o status, aciona o LED caso a lixeira esteja cheia, monta a mensagem JSON e publica os dados no tópico MQTT. Também é exibida no Monitor Serial a mensagem enviada, permitindo acompanhar o funcionamento do firmware durante os testes.

2.12 Processamento dos dados no Node-RED e dashboard

O Node-RED foi utilizado como plataforma de integração entre o broker MQTT e o dashboard de monitoramento. O fluxo desenvolvido recebe as mensagens MQTT, converte os dados em JSON, processa as informações e atualiza automaticamente os elementos visuais.

O Node-RED é amplamente utilizado em aplicações de Internet das Coisas por permitir a integração de dispositivos, serviços e protocolos por meio de programação baseada em fluxos, simplificando a construção de dashboards e sistemas de monitoramento (NODE-RED, 2026).

O dashboard apresenta quatro informações principais: nível de preenchimento, status atual da lixeira, histórico das medições e horário da última atualização. O nível de preenchimento é exibido em formato percentual, facilitando a interpretação pelo usuário.

Como a leitura física do sensor é baseada em distância, foi necessário converter essa informação para percentual. A lógica aplicada considera que quanto menor a distância, maior é o nível de preenchimento. Dessa forma, o dashboard apresenta a informação de maneira mais intuitiva.

Tabela 3 – Critérios de classificação visual no dashboard.

Percentual de preenchimento	Classificação visual
0% a 45%	VAZIA
45% a 75%	PARCIAL
75% a 100%	CHEIA

Fonte: Elaborado pelos autores (2026).

O indicador gráfico apresenta o percentual de preenchimento com cores diferentes para cada faixa. O status textual exibe a condição atual da lixeira. O gráfico histórico registra a variação das medições ao longo do tempo. O campo de última atualização mostra a data e o horário da última mensagem recebida, permitindo verificar se a comunicação permanece ativa.

2.13 Fluxograma de funcionamento do protótipo

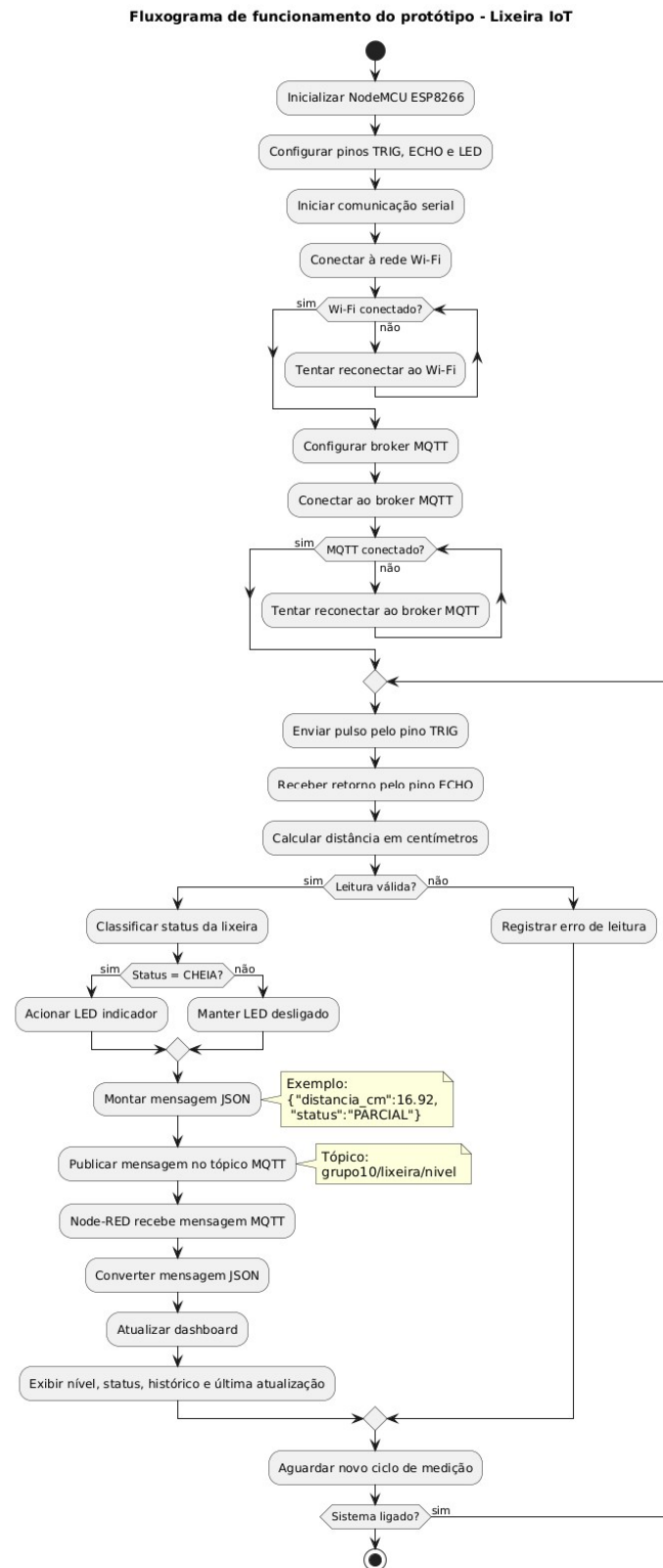


Figura 5 - Fluxograma de funcionamento do protótipo.

Fonte: Elaborado pelos autores (2026).

3. Resultados

O protótipo final da Lixeira IoT foi implementado fisicamente utilizando NodeMCU ESP8266, sensor ultrassônico HC-SR04, LED indicador, resistores e montagem em protoboard. O sistema foi testado por meio da aproximação e afastamento de objetos em relação ao sensor, simulando diferentes níveis de preenchimento da lixeira.

Durante os testes, o sensor realizou medições contínuas da distância entre sua posição e o objeto detectado. Esses valores foram processados pelo NodeMCU, classificados conforme os limites definidos no firmware e enviados ao broker MQTT no formato JSON. O Node-RED recebeu as mensagens em tempo real e atualizou automaticamente o dashboard.

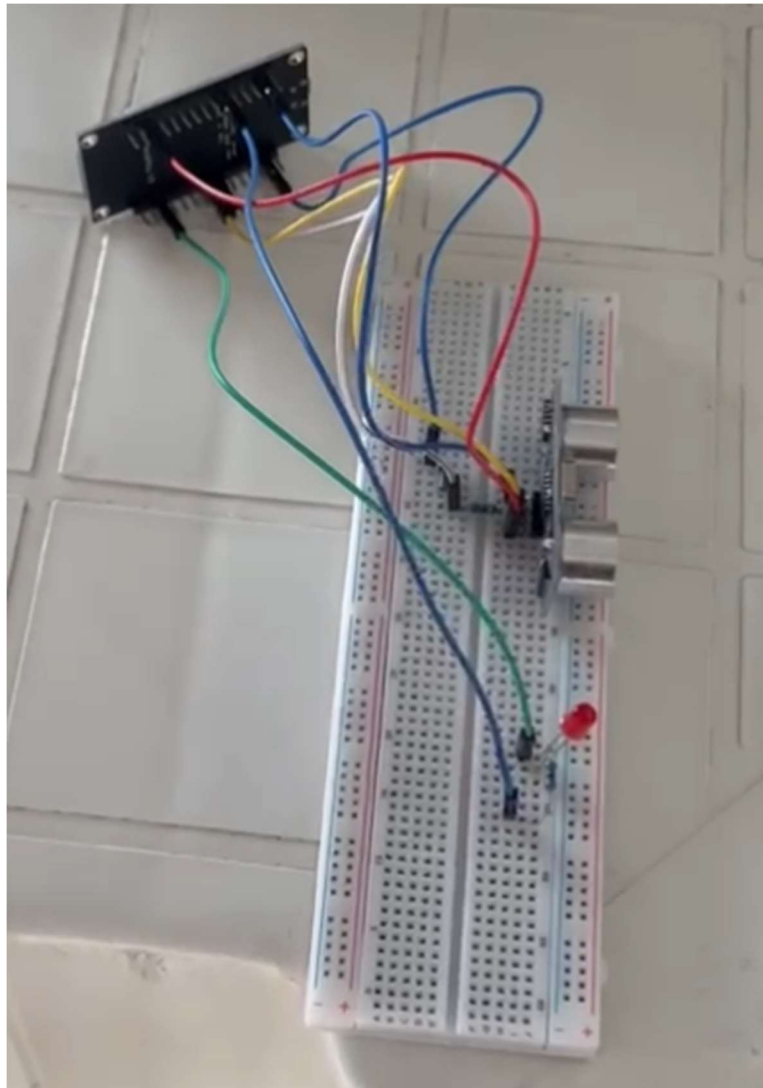


Figura 6 - Protótipo físico da Lixeira IoT.

Fonte: Elaborado pelos autores (2026).

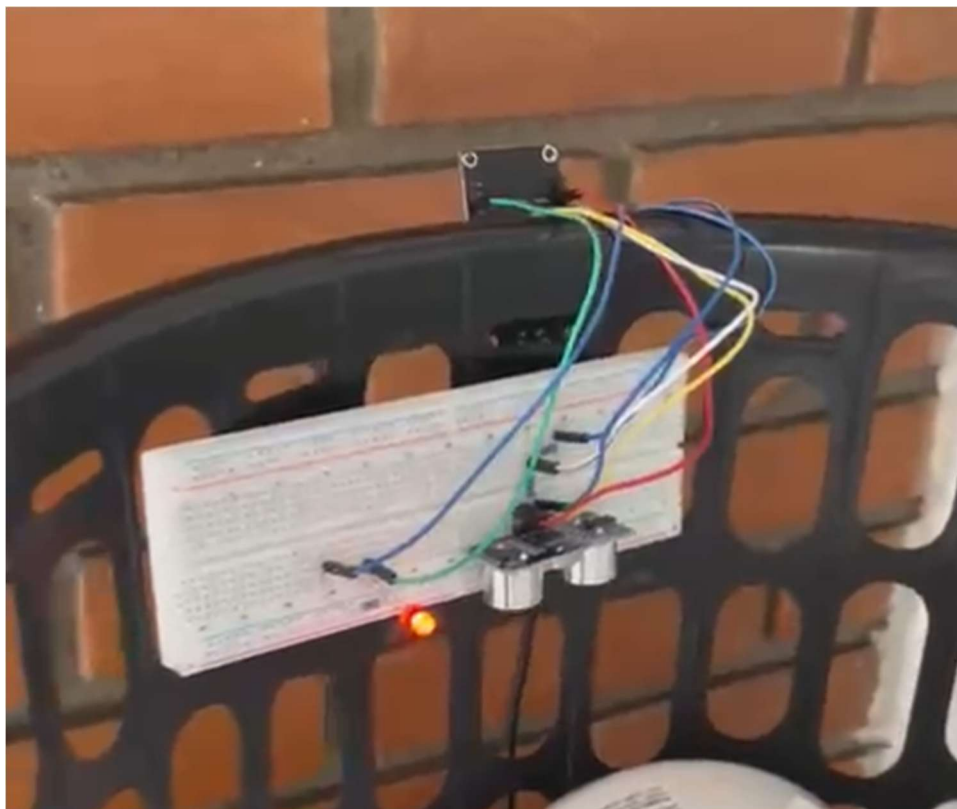


Figura 7 - Protótipo físico da Lixeira IoT em funcionamento.

Fonte: Elaborado pelos autores (2026).

```
Saída  Monitor Serial  X
Mensagem (NodeMCU 1.0 (ESP-12E Module) + Enter para enviar mensagem para 'COM14' em '{2}'

{"distancia_cm":16.92,"status":"PARCIAL"}
{"distancia_cm":16.93,"status":"PARCIAL"}
{"distancia_cm":16.92,"status":"PARCIAL"}
{"distancia_cm":16.92,"status":"PARCIAL"}
{"distancia_cm":16.93,"status":"PARCIAL"}
{"distancia_cm":16.93,"status":"PARCIAL"}
{"distancia_cm":16.92,"status":"PARCIAL"}
{"distancia_cm":16.93,"status":"PARCIAL"}
```

Figura 8 - Monitor Serial exibindo distância medida.

Fonte: Elaborado pelos autores a partir do Arduino IDE (2026)

```
30/05/2026, 13:10:19 node: debug 1
grupo10/lixreira/nivel : msg.payload : Object
  ▸ { distancia_cm: 6.51, status:
    "CHEIA" }

30/05/2026, 13:10:21 node: debug 1
grupo10/lixreira/nivel : msg.payload : Object
  ▸ { distancia_cm: 6.51, status:
    "CHEIA" }

30/05/2026, 13:10:23 node: debug 1
grupo10/lixreira/nivel : msg.payload : Object
  ▸ { distancia_cm: 6.51, status:
    "CHEIA" }

30/05/2026, 13:10:25 node: debug 1
grupo10/lixreira/nivel : msg.payload : Object
  ▸ { distancia_cm: 6.51, status:
    "CHEIA" }
```

Figura 9 - Painel Debug do Node-RED exibindo mensagens JSON recebidas via MQTT.

Fonte: Elaborado pelos autores (2026).

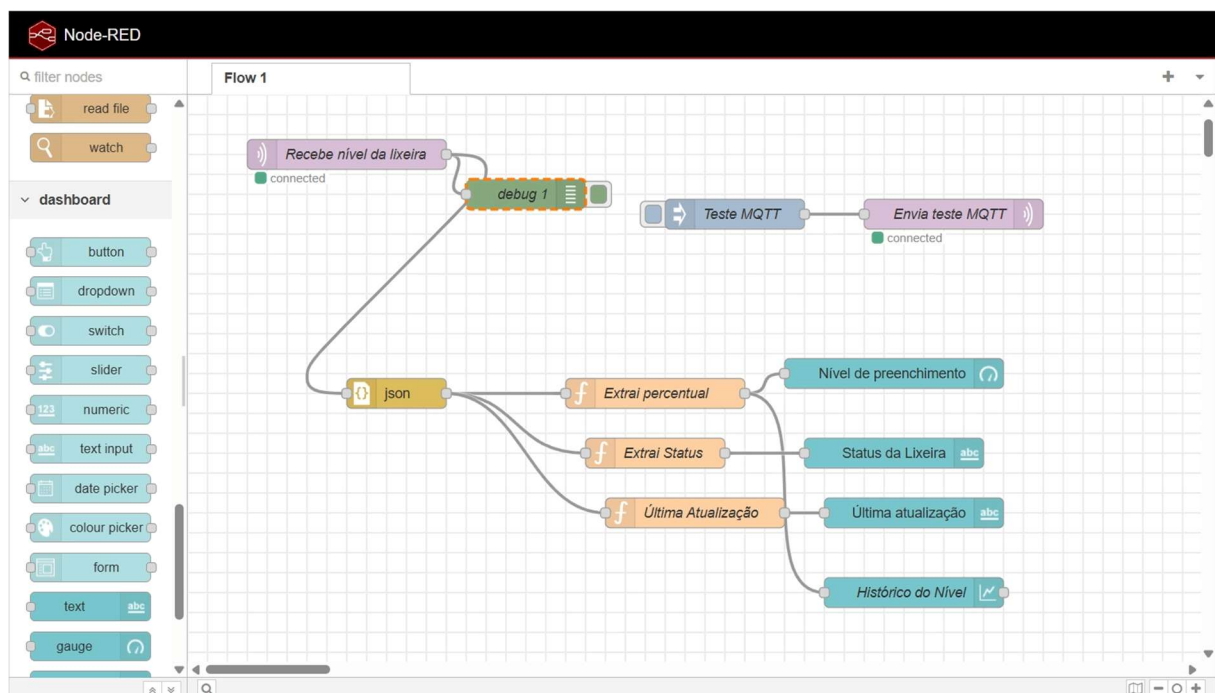


Figura 10 - Fluxo do Node-RED responsável pelo recebimento e tratamento das mensagens MQTT.

Fonte: Elaborado pelos autores (2026).



Figura 11 - Dashboard final com nível de preenchimento, status, histórico e última atualização.

Fonte: Elaborado pelos autores (2026).

Os testes demonstraram que o sistema foi capaz de identificar diferentes condições de preenchimento. Quando o objeto foi posicionado próximo ao sensor, a distância medida diminuiu e o sistema classificou a lixeira como cheia. Em distâncias intermediárias, a classificação obtida foi parcial. Com maior distância entre o sensor e o objeto, o sistema classificou a lixeira como vazia.

Tabela 4 – Testes funcionais do protótipo.

Situação simulada	Distância esperada	Status esperado	Resultado obtido
Objeto próximo ao sensor	Menor que 10 cm	CHEIA	Aprovado
Objeto em distância intermediária	Entre 10 cm e 35 cm	PARCIAL	Aprovado
Objeto afastado do sensor	Maior que 35 cm	VAZIA	Aprovado
Comunicação MQTT ativa	Mensagem recebida no Node-RED	Dashboard atualizado	Aprovado

Fonte: Elaborado pelos autores (2026).

3.1 Medições de tempo de resposta

Para avaliar o desempenho do protótipo, foram realizadas medições do tempo de resposta entre a detecção realizada pelo sensor e o recebimento dos dados na plataforma MQTT/Node-RED. Também foram realizadas medições do tempo entre a identificação da condição de lixeira cheia e o acionamento do LED.

As medições foram realizadas manualmente durante os testes do protótipo, considerando quatro repetições para o sensor e quatro repetições para o atuador.

Tabela 5 – Tempo de resposta do sensor.

Número da medida	Sensor	Evento avaliado	Tempo de resposta
1	Sensor HC-SR04	Detecção até recebimento no MQTT/Node-RED	2s
2	Sensor HC-SR04	Detecção até recebimento no MQTT/Node-RED	2s
3	Sensor HC-SR04	Detecção até recebimento no MQTT/Node-RED	3s
4	Sensor HC-SR04	Detecção até recebimento no MQTT/Node-RED	3s
Média	Sensor HC-SR04	Tempo médio de resposta	2,5s

Fonte: Elaborado pelos autores (2026).

Tabela 6 – Tempo de resposta do atuador.

Número da medida	Atuador	Evento avaliado	Tempo de resposta
1	LED indicador	Classificação CHEIA até acionamento do LED	4s
2	LED indicador	Classificação CHEIA até acionamento do LED	5s
3	LED indicador	Classificação CHEIA até acionamento do LED	5s
4	LED indicador	Classificação CHEIA até acionamento do LED	6s
Média	LED indicador	Tempo médio de resposta	5s

Fonte: Elaborado pelos autores (2026).

O tempo de resposta do sensor ficou próximo ao intervalo configurado no firmware, que realiza publicações a cada dois segundos. Já o tempo de resposta do LED apresentou média maior, pois a medição considerou o intervalo entre a alteração física simulada no sensor, a nova leitura válida do sistema e a identificação da condição CHEIA.

A média foi calculada pela expressão:

$$\text{Tempo médio} = (M1 + M2 + M3 + M4) / 4$$

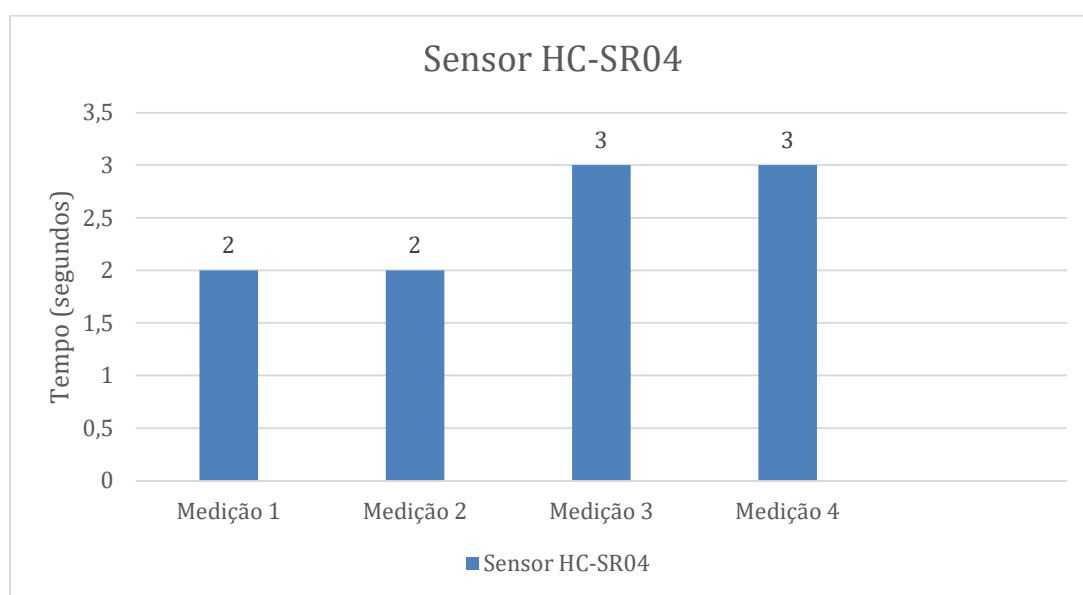


Figura 12 – Gráfico de tempo de resposta do sensor HC-SR04 até o recebimento no MQTT/Node-RED.

Fonte: Elaborado pelos autores (2026).

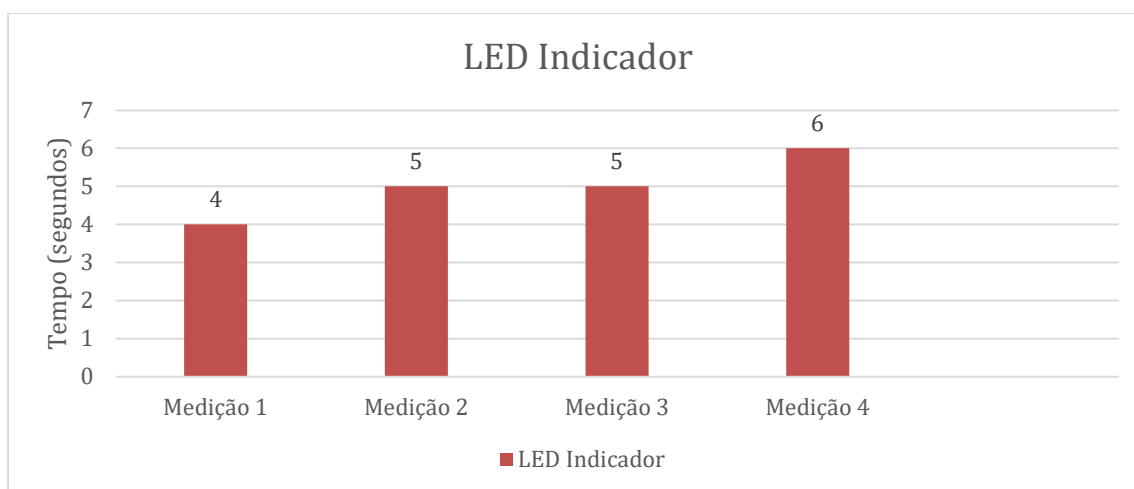


Figura 13 – Gráfico de tempo de resposta do LED indicador

Fonte: Elaborado pelos autores (2026).

Os resultados das medições permitem avaliar a estabilidade do protótipo. Como o firmware foi configurado para publicar dados a cada dois segundos, espera-se que a atualização no dashboard ocorra dentro desse intervalo. Para a aplicação proposta, esse tempo é adequado, pois o monitoramento do nível de resíduos não exige resposta instantânea em milissegundos, mas sim atualização periódica e confiável.

3.2 Vídeo-demonstração e repositório

O vídeo-demonstração do projeto foi publicado no YouTube em modo não listado. No vídeo, são apresentados o protótipo físico, o funcionamento do sensor, o acionamento do LED, o envio de dados via MQTT, o fluxo no Node-RED e o dashboard em funcionamento.

Link do vídeo: <https://www.youtube.com/watch?v=0NhH5E9Q5Mw>

O repositório do projeto foi disponibilizado no GitHub e contém o código-fonte, a documentação do software, a descrição do hardware utilizado, o diagrama de montagem, as interfaces de comunicação e as informações necessárias para reprodução do protótipo.

Link do GitHub: <https://github.com/CauePaganeli/Lixeira-Inteligente-IOT---OIC---G10>

3.3 Principais problemas enfrentados

Durante o desenvolvimento do protótipo, um dos principais desafios foi a compatibilidade elétrica entre o sensor ultrassônico HC-SR04 e o NodeMCU ESP8266. O sensor pode retornar sinal de 5 V no pino ECHO, enquanto o ESP8266 opera com lógica de 3,3 V. Para solucionar esse problema, foi utilizado um divisor de tensão com resistores, protegendo a entrada digital do microcontrolador.

Outro desafio foi a montagem física na protoboard, pois a placa NodeMCU ocupa grande parte dos espaços disponíveis. Para contornar essa limitação, parte das ligações foi organizada fora da área central da protoboard, mantendo as conexões do sensor, resistores e LED de forma funcional e segura.

Também houve necessidade de ajustar a lógica de interpretação dos dados. A leitura física do sensor é baseada em distância, mas a interface do usuário é mais intuitiva quando apresentada

em percentual. Por isso, o firmware manteve a classificação por distância, enquanto o Node-RED realizou a conversão da distância em percentual para exibição no dashboard.

A comunicação MQTT também exigiu validação. Inicialmente, foi necessário garantir que o NodeMCU estivesse conectado ao Wi-Fi, ao broker MQTT e publicando corretamente no tópico configurado. O Monitor Serial e o painel Debug do Node-RED foram utilizados para validar o envio e o recebimento das mensagens.

4. Conclusões

Os objetivos propostos foram alcançados, pois o protótipo conseguiu medir o nível de preenchimento da lixeira, classificar seu estado, acionar um atuador visual e transmitir os dados pela internet utilizando o protocolo MQTT. A integração entre NodeMCU ESP8266, sensor ultrassônico HC-SR04, LED, broker MQTT, Node-RED e dashboard demonstrou o funcionamento de uma solução IoT completa.

Os principais problemas enfrentados estiveram relacionados à compatibilidade elétrica entre sensor e microcontrolador, à limitação física da montagem na protoboard, à interpretação da distância como nível de preenchimento e à configuração da comunicação MQTT. Esses problemas foram resolvidos com o uso de divisor de tensão, reorganização das conexões, conversão da distância em percentual no Node-RED e validação das mensagens por meio do Monitor Serial e do Debug do Node-RED.

Entre as vantagens do projeto, destacam-se o baixo custo dos componentes, a facilidade de reprodução, a comunicação via internet, a visualização em tempo real e a possibilidade de expansão para múltiplas lixeiras. O uso do NodeMCU simplificou a arquitetura por já possuir Wi-Fi integrado, reduzindo a necessidade de módulos adicionais.

Como desvantagens, o protótipo depende de uma rede Wi-Fi disponível, utiliza um broker público para testes, não possui encapsulamento definitivo para uso externo e pode sofrer variações de leitura conforme o posicionamento do sensor ou o tipo de resíduo presente na lixeira.

Como melhorias futuras, recomenda-se desenvolver uma estrutura física mais robusta, utilizar alimentação dedicada ou bateria, implementar armazenamento histórico em banco de dados, configurar um broker MQTT privado com autenticação, adicionar alertas por e-mail ou

aplicativo e expandir o sistema para o monitoramento simultâneo de várias lixeiras em um painel centralizado.

5. Referências

ANANDRAVISEKAR, G. et al. **IoT Based Surveillance Robot**. *International Journal of Engineering Research & Technology (IJERT)*, v. 7, n. 3, 2018.

BYERS, C. **Sensores e atuadores IoT**. *Embarcados*, [s.d.]. Disponível em: <https://www.embarcados.com.br/sensores-e-atuadores-iot/>. Acesso em: 28 abr. 2026.

FRITZING. **Fritzing**. Disponível em: <https://fritzing.org/>. Acesso em: 28 abr. 2026.

HC-SR04 ULTRASONIC SENSOR DATASHEET. **HC-SR04 Ultrasonic Sensor Datasheet**. Disponível em: <https://cdn.sparkfun.com/datasheets/Sensors/Proximity/HCSR04.pdf>. Acesso em: 28 abr. 2026.

IBM. **Why MQTT is ideal for IoT**. Disponível em: <https://www.ibm.com/developerworks/br/library/iot-mqtt-why-good-for-iot/index.html>. Acesso em: 28 abr. 2026.

MQTT.ORG. **MQTT Version 3.1.1**. Disponível em: <http://mqtt.org/>. Acesso em: 28 abr. 2026.

NODEMCU TEAM. **NodeMCU**. Disponível em: https://www.nodemcu.com/index_en.html. Acesso em: 28 abr. 2026.

OLIVEIRA, A. S.; ANDRADE, F. S. **Sistemas embarcados: hardware e firmware na prática**. São Paulo: Érica, 2009.

ROBOCORE. **Sensor de distância ultrassônico HC-SR04**. Disponível em: <https://www.robocore.net/sensor-robo/sensor-de-distancia-ultrassonico-hc-sr04>. Acesso em: 28 abr. 2026.

SANTOS, B. P. et al. **Internet das Coisas: da teoria à prática**. Disponível em: <https://homepages.dcc.ufmg.br/~mmvieira/cc/papers/internet-das-coisas.pdf>. Acesso em: 28 abr. 2026.

ARDUINO. **Arduino IDE**. Disponível em: <https://www.arduino.cc/en/software>. Acesso em: 30 maio 2026.

HIVEMQ. **Public MQTT Broker**. Disponível em: <https://www.hivemq.com/mqtt/public-mqtt-broker/>. Acesso em: 30 maio 2026.

NODE-RED. **Node-RED: low-code programming for event-driven applications.**

Disponível em: <https://nodered.org/>. Acesso em: 30 maio 2026.

NAYYAR, Anand; PURI, Vikram. **Taking MQTT and NodeMCU to IoT: Communication in Internet of Things.** Procedia Computer Science, v. 132, p. 1611-1618, 2018. Disponível

em: <https://doi.org/10.1016/j.procs.2018.05.126>. Acesso em: 30 maio 2026.